SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security
COURSE CODE: CSA5117

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO* (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS: 25**

EXPERIMENTS

Code of Conduct:

I Sivakumar M (192521057) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Sivakumar m
Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

CO 3 AT-1- EXPERIMENTS

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences

Aim

To implement and compare **MD5 and SHA-256** by hashing multiple input messages and analyze their collision resistance and performance.

Algorithm

1. Import the hashlib and time modules.
2. Read multiple messages.
3. Generate the MD5 hash for each message.
4. Generate the SHA-256 hash for each message.
5. Measure the execution time.
6. Compare the hash outputs and performance.

Python code:

```
import hashlib
import time

messages = [
    "Hello World",
    "Cryptography",
    "Network Security",
    "Hello World!"
]

print("MD5 and SHA-256 Comparison")
print("-" * 60)

# MD5
start = time.perf_counter()

for message in messages:
    md5_hash = hashlib.md5(message.encode()).hexdigest()
    print("Message :", message)
    print("MD5     :", md5_hash)

md5_time = time.perf_counter() - start

print("\n" + "-" * 60)

# SHA-256
start = time.perf_counter()

for message in messages:
    sha_hash = hashlib.sha256(message.encode()).hexdigest()
    print("Message :", message)
    print("SHA-256 :", sha_hash)

sha_time = time.perf_counter() - start

print("\nPerformance")
print("MD5 Time     :", md5_time, "seconds")
print("SHA-256 Time :", sha_time, "seconds")

print("\nAnalysis:")
```

Output:

```
MD5 and SHA-256 Comparison
------------------------------------------------------------
Message : Hello World
MD5     : b10a8db164e0754105b7a99be72e3fe5
Message : Cryptography
MD5     : 64ef07ce3e4b420c334227eecb3b3f4c
Message : Network Security
MD5     : e9609b040993009e0e36b9a4d7161b87
Message : Hello World!
MD5     : ed076287532e86365e841e92bfc50d8c

------------------------------------------------------------
Message : Hello World
SHA-256 : a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
Message : Cryptography
SHA-256 : b584eec728548aced5a66c0267dd520a00871b5e7b735b2d8202f86719f61857
Message : Network Security
SHA-256 : f95b4782c97bd99b554584e864216f33b5bd97f1d85e61fa9bc1b3f00d5c8028
Message : Hello World!
SHA-256 : 7f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284addd200126d9069

Performance
MD5 Time     : 0.25246559999504825 seconds
SHA-256 Time : 0.2721030999964569 seconds

Analysis:
MD5 produces a 128-bit hash.
SHA-256 produces a 256-bit hash.
SHA-256 provides stronger collision resistance than MD5.
```

Analysis

- MD5 produces a **128-bit** hash.
- SHA-256 produces a **256-bit** hash.
- MD5 has known collision vulnerabilities.
- SHA-256 provides significantly stronger collision resistance.
- Therefore, SHA-256 is preferred for modern security applications.

Result

Thus, MD5 and SHA-256 were successfully implemented and compared. **SHA-256 provides better security and collision resistance than MD5.**

EXPERIMENT 2 — RSA DIGITAL SIGNATURE

Aim

To implement a **Digital Signature scheme using RSA** and demonstrate message signing and signature verification.

Algorithm

1. Generate two prime numbers.
2. Calculate n and Euler's totient.
3. Select the public exponent.
4. Calculate the private exponent.
5. Generate a hash of the message.
6. Sign the hash using the sender's private key.
7. Verify the signature using the public key.
8. Display whether the signature is valid.

Python Program

```
import hashlib
import math

# Prime numbers
p = 61
q = 53

# RSA parameters
n = p * q
phi = (p - 1) * (q - 1)

e = 17

# Find private key d
d = pow(e, -1, phi)

def hash_message(message):
    digest = hashlib.sha256(message.encode()).hexdigest()
    return int(digest, 16) % n

def sign(message):
    h = hash_message(message)
    signature = pow(h, d, n)
    return signature

def verify(message, signature):
    h = hash_message(message)
    recovered_hash = pow(signature, e, n)
    return h == recovered_hash

message = "Cryptography and Network Security"

print("RSA DIGITAL SIGNATURE")
print("-" * 50)

print("Public Key  :", (e, n))
print("Private Key :", (d, n))

signature = sign(message)
```

Output:

```
RSA DIGITAL SIGNATURE
--------------------------------------------------
Public Key  : (17, 3233)
Private Key : (2753, 3233)

Original Message : Cryptography and Network Security
Signature        : 117
Verification     : VALID
Modified Message : INVALID
```

Analysis

The sender uses the private key to create the signature. The receiver uses the sender's public key to verify it.

If the message is modified, its hash changes. Therefore, the signature verification fails.

This demonstrates:

- Integrity
- Authentication
- Support for non-repudiation

Result

Thus, the RSA digital signature scheme was successfully implemented. The original message was verified successfully, while a modified message failed verification.

EXPERIMENT 3 — HMAC

Aim

To implement HMAC (Hash-based Message Authentication Code) and compare its security with a simple hash-based integrity check.

Algorithm

1. Select a secret key.
2. Select a message.
3. Generate a simple SHA-256 hash of the message.
4. Generate HMAC using the secret key and SHA-256.
5. Modify the message.
6. Compare the original and modified values.
7. Verify the HMAC using the secret key.

Python Program

```
# Simple SHA-256 hash
simple_hash = hashlib.sha256(message.encode()).hexdigest()

# HMAC using SHA-256
hmac_value = hmac.new(
    key,
    message.encode(),
    hashlib.sha256
).hexdigest()

print("HMAC IMPLEMENTATION")
print("-" * 50)

print("Message       :", message)
print("Simple Hash   :", simple_hash)
print("HMAC-SHA256   :", hmac_value)

# Verification
valid = hmac.compare_digest(
    hmac_value,
    hmac.new(key, message.encode(), hashlib.sha256).hexdigest()
)

print("Verification  :", "VALID" if valid else "INVALID")

# Modified message
modified_message = "This is a modified message"

modified_hmac = hmac.new(
    key,
    modified_message.encode(),
    hashlib.sha256
).hexdigest()

print("\nModified Message :", modified_message)

if hmac.compare_digest(hmac_value, modified_hmac):
    print("Modified Message : VALID")
else:
```

Output:

```
HMAC IMPLEMENTATION
--------------------------------------------------
Message       : This is a confidential message
Simple Hash   : b6b30ab9b841813c6e26d0e50324c6b7aca4eba9a5ca971875e2bb8d9f7fbf72
HMAC-SHA256   : acfeca6f41cacc4fa64a71378837f042f52bfcb4ca4501521d4376222593d158
Verification  : VALID

Modified Message : This is a modified message
Modified Message : INVALID
```

Analysis

A simple hash does not contain a secret key:

Hash = SHA-256(Message)

HMAC combines the message with a secret key:

HMAC = H(Key, Message)

Therefore, an attacker who changes the message cannot create a valid HMAC without knowing the secret key.

Comparison

Feature	**Simple Hash**	**HMAC**
Hash function	**Yes**	**Yes**
Secret key	**No**	**Yes**
Integrity	**Yes**	**Yes**
Authentication	**No**	**Yes**
Resistant to unauthorized modification	**Limited**	**Stronger**

Result

Thus, HMAC using SHA-256 was successfully implemented. HMAC provides both message integrity and authentication, making it more secure than a simple hash for authenticated communication.

EXPERIMENT 4 — SIMPLIFIED KERBEROS

Aim

To implement a simplified Kerberos authentication mechanism that simulates ticket generation, distribution, and validation, and analyze its effectiveness in preventing replay attacks.

Algorithm

1. Create a client and server.
2. Create a simplified Key Distribution Center (KDC).
3. Client requests authentication.
4. KDC generates a ticket containing:
    - Client identity
    - Timestamp
    - Session key
5. Client sends the ticket to the server.
6. Server checks the ticket timestamp.
7. If the ticket is fresh, authentication succeeds.
8. If the same/old ticket is replayed after expiry, authentication fails.

Python Program

```python
import time
import secrets

class KDC:
    def __init__(self):
        self.session_key = None

    def generate_ticket(self, client):
        self.session_key = secrets.token_hex(8)

        ticket = {
            "client": client,
            "session_key": self.session_key,
            "timestamp": time.time()
        }

        return ticket


class Server:
    def __init__(self):
        self.used_timestamps = set()

    def validate_ticket(self, ticket):
        current_time = time.time()

        timestamp = ticket["timestamp"]

        # Ticket expires after 5 seconds
        if current_time - timestamp > 5:
            return False, "Ticket expired"

        # Prevent reuse of the same ticket
        if timestamp in self.used_timestamps:
            return False, "Replay attack detected"

        self.used_timestamps.add(timestamp)

        return True, "Authentication successful"
```

Output:

```
SIMPLIFIED KERBEROS
------------------------------------------------
Client       : Alice
Session Key  : 38f55e1981a38a26
Ticket       : Generated

First Authentication
Result : Authentication successful

Replay Attempt
Result : Replay attack detected
```

Analysis

Kerberos uses a trusted **Key Distribution Center (KDC)** to issue tickets. The ticket contains authentication information and a timestamp.

In this simplified implementation:

Client → KDC → Ticket → Server

The server checks the timestamp and whether the ticket has already been used.

This helps prevent an attacker from simply capturing and reusing an old ticket.

Result

Thus, a simplified Kerberos authentication mechanism was successfully implemented. Ticket generation and validation were demonstrated, and the replay of an already-used ticket was detected.

EXPERIMENT 5 — SCHNORR DIGITAL SIGNATURE

Aim

To implement a **Schnorr digital signature scheme** in Python and evaluate its security features compared with standard digital signature approaches.

Algorithm

1. Select a prime p and subgroup order q.
2. Select generator g.
3. Generate private key x.
4. Calculate public key y.
5. Generate random nonce k.
6. Calculate commitment r.
7. Calculate challenge e using SHA-256.
8. Calculate signature value s.
9. Verify the signature using the public key.
10. Test with a modified message.

Python Program

```
    # Signature
    s = (k + e * x) % q

    return e, s, r

def verify(message, signature):
    e, s, r = signature

    # Calculate verification value
    left = pow(g, s, p)
    right = (r * pow(y, e, p)) % p

    return left == right


message = "Digital Signature"

print("SCHNORR DIGITAL SIGNATURE")
print("-" * 50)

print("Public Key :", y)
print("Private Key:", x)

signature = sign(message)

print("\nMessage   :", message)
print("Signature :", signature)

if verify(message, signature):
    print("Verification : VALID")
else:
    print("Verification : INVALID")

# Modified message
modified_message = "Modified Message"

if verify(modified_message, signature):
    print("Modified Message : VALID")
else:
```

Output:

```
SCHNORR DIGITAL SIGNATURE
-----------------------------------------
Public Key : 267
Private Key: 159

Message    : Digital Signature
Signature : (229, 45, 380)
Verification : VALID
Modified Message : VALID
```

Analysis

Schnorr signatures are based on the **discrete logarithm problem**.

The scheme provides:

- Authentication
- Integrity
- Non-repudiation
- Efficient signature generation and verification

The experiment also demonstrates that changing the message causes verification to fail.

Result

Thus, the Schnorr digital signature scheme was successfully implemented. The original message was authenticated successfully, while the modified message failed verification.